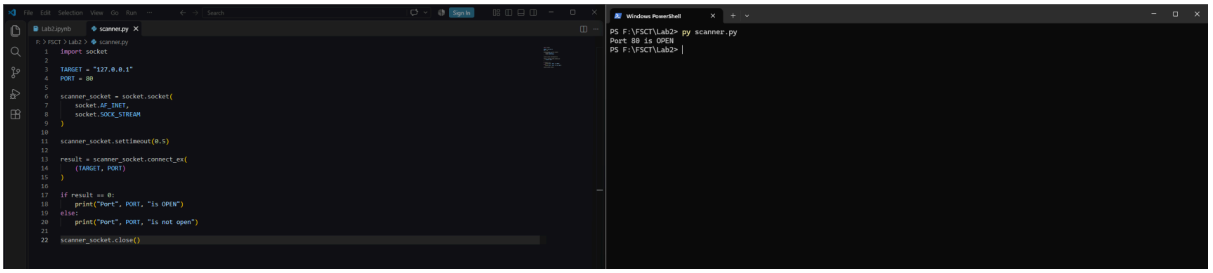
Part 3



```
import socket

TARGET = "127.0.0.1"
PORT = 80

scanner_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

scanner_socket.settimeout(0.5)

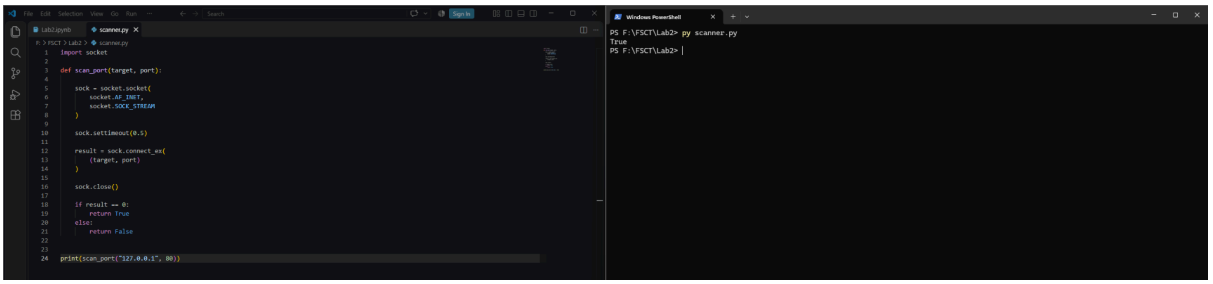
result = scanner_socket.connect_ex(
    (TARGET, PORT)
)

if result == 0:
    print("Port", PORT, "is OPEN")
else:
    print("Port", PORT, "is not open")

scanner_socket.close()
```

```
PS F:\FSCT\Lab2> py scanner.py
Port 80 is OPEN
PS F:\FSCT\Lab2>
```

Part 4



```
import socket

def scan_port(target, port):

    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    sock.settimeout(0.5)

    result = sock.connect_ex(
        (target, port)
    )

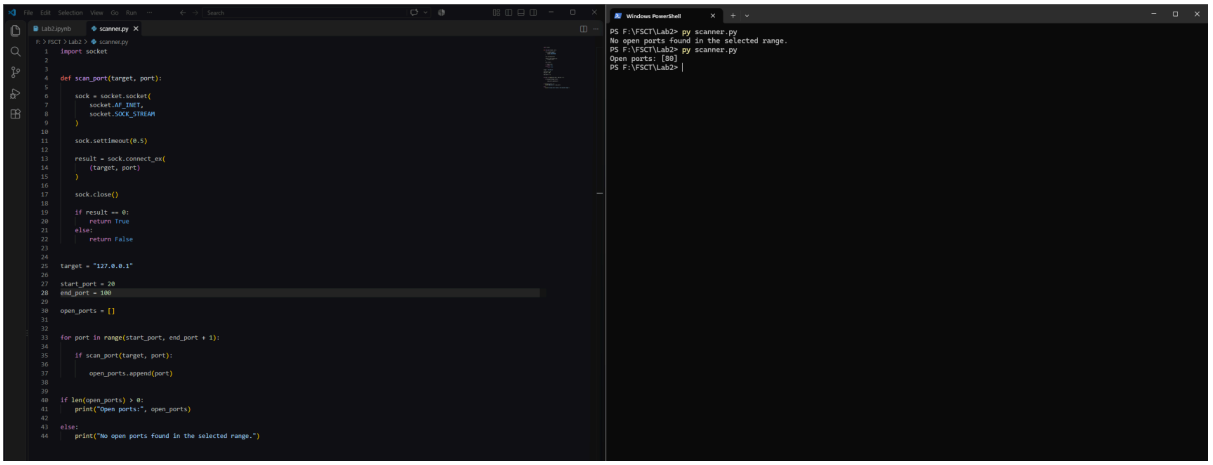
    sock.close()

    if result == 0:
        return True
    else:
        return False


print(scan_port("127.0.0.1", 80))
```

```
PS F:\FSCT\Lab2> py scanner.py
True
PS F:\FSCT\Lab2>
```

Part 5



```
import socket

def scan_port(target, port):

    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    sock.settimeout(0.5)

    result = sock.connect_ex(
        (target, port)
    )

    sock.close()

    if result == 0:
        return True
    else:
        return False


target = "127.0.0.1"

start_port = 20
end_port = 100

open_ports = []


for port in range(start_port, end_port + 1):

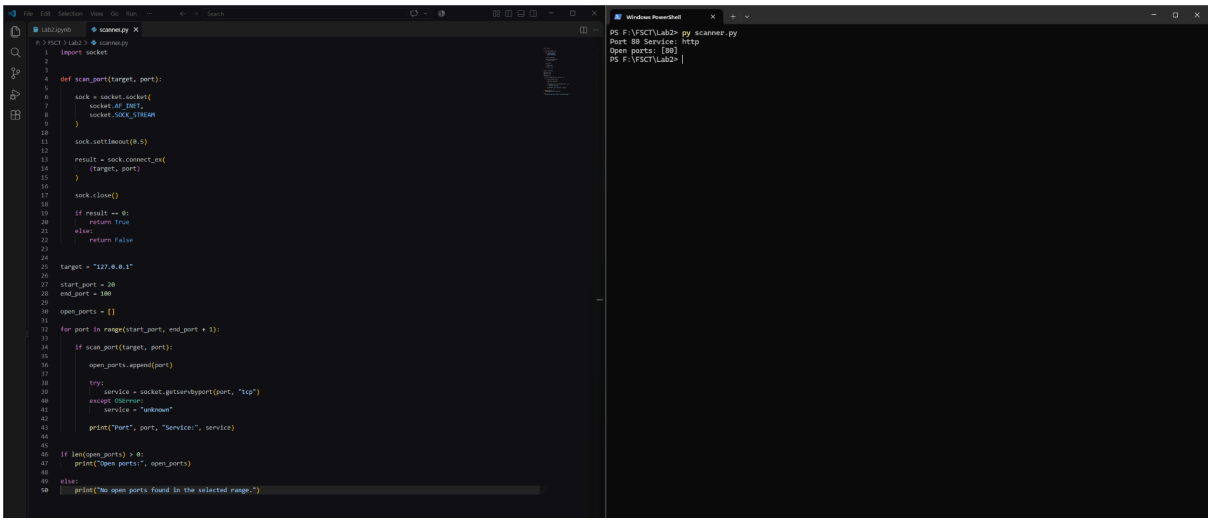
    if scan_port(target, port):

        open_ports.append(port)


if len(open_ports) > 0:
    print("Open ports:", open_ports)
else:
    print("No open ports found in the selected range.")
```

```
PS F:\FSCT\Lab2> py scanner.py
No open ports found in the selected range.
PS F:\FSCT\Lab2> py scanner.py
Open ports: [80]
PS F:\FSCT\Lab2>
```

Part 6



```
import socket


def scan_port(target, port):

    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    sock.settimeout(0.5)

    result = sock.connect_ex(
        (target, port)
    )

    sock.close()

    if result == 0:
        return True
    else:
        return False


target = "127.0.0.1"

start_port = 20
end_port = 100

open_ports = []

for port in range(start_port, end_port + 1):

    if scan_port(target, port):

        open_ports.append(port)

        try:
            service = socket.getservbyport(port, "tcp")
        except OSError:
            service = "unknown"

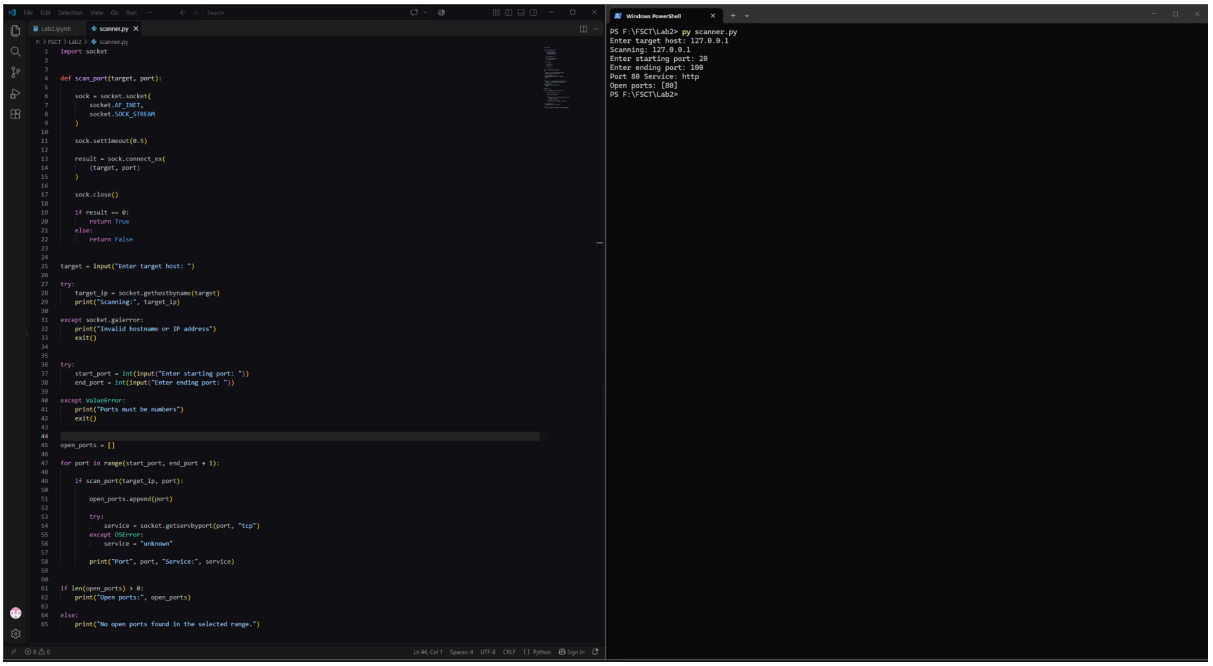
        print("Port", port, "Service:", service)


if len(open_ports) > 0:
    print("Open ports:", open_ports)

else:
    print("No open ports found in the selected range.")
```

```
PS F:\FSCT\Lab2> py scanner.py
Port 80 Service: http
Open ports: [80]
PS F:\FSCT\Lab2>
```

Part 7



```
import socket

def scan_port(target, port):

    sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_STREAM
    )

    sock.settimeout(0.5)

    result = sock.connect_ex(
        (target, port)
    )

    sock.close()

    if result == 0:
        return True
    else:
        return False


target = input("Enter target host: ")

try:
    target_ip = socket.gethostbyname(target)
    print("Scanning:", target_ip)

except socket.gaierror:
    print("Invalid hostname or IP address")
    exit()


try:
    start_port = int(input("Enter starting port: "))
    end_port = int(input("Enter ending port: "))

except ValueError:
    print("Ports must be numbers")
    exit()

open_ports = []

for port in range(start_port, end_port + 1):

    if scan_port(target_ip, port):

        open_ports.append(port)

        try:
            service = socket.getservbyport(port, "tcp")
        except OSError:
            service = "unknown"

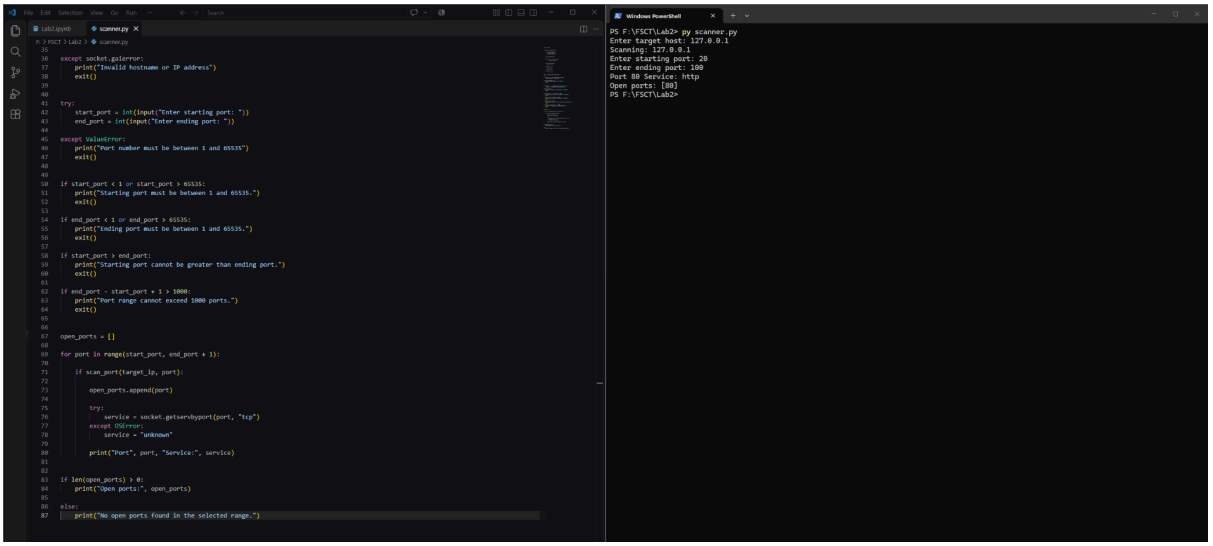
        print("Port", port, "Service:", service)


if len(open_ports) > 0:
    print("Open ports:", open_ports)

else:
    print("No open ports found in the selected range.")
```

```
PS F:\FSCT\Lab2> py scanner.py
Enter target host: 127.0.0.1
Scanning: 127.0.0.1
Enter starting port: 20
Enter ending port: 100
Port 80 Service: http
Open ports: [80]
PS F:\FSCT\Lab2>
```

Part 8/9



```
except socket.gaierror:
    print("Invalid hostname or IP address")
    exit()


try:
    start_port = int(input("Enter starting port: "))
    end_port = int(input("Enter ending port: "))

except ValueError:
    print("Port number must be between 1 and 65535")
    exit()


if start_port < 1 or start_port > 65535:
    print("Starting port must be between 1 and 65535.")
    exit()

if end_port < 1 or end_port > 65535:
    print("Ending port must be between 1 and 65535.")
    exit()

if start_port > end_port:
    print("Starting port cannot be greater than ending port.")
    exit()

if end_port - start_port + 1 > 1000:
    print("Port range cannot exceed 1000 ports.")
    exit()


open_ports = []

for port in range(start_port, end_port + 1):

    if scan_port(target_ip, port):

        open_ports.append(port)

        try:
            service = socket.getservbyport(port, "tcp")
        except OSError:
            service = "unknown"

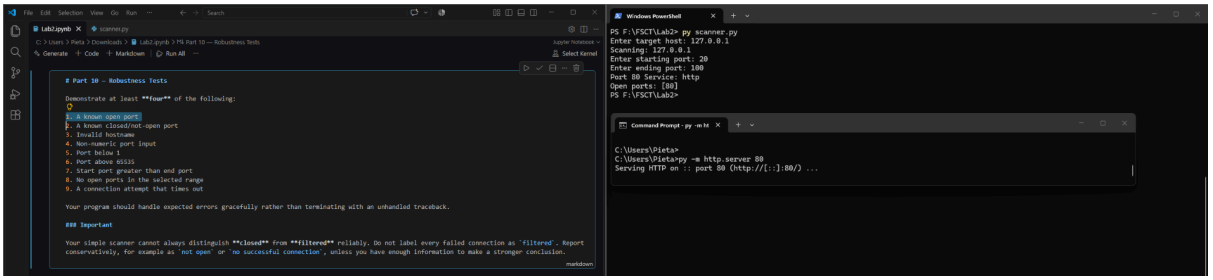
        print("Port", port, "Service:", service)


if len(open_ports) > 0:
    print("Open ports:", open_ports)

else:
    print("No open ports found in the selected range.")
```

```
PS F:\FSCT\Lab2> py scanner.py
Enter target host: 127.0.0.1
Scanning: 127.0.0.1
Enter starting port: 20
Enter ending port: 100
Port 80 Service: http
Open ports: [80]
PS F:\FSCT\Lab2>
```

Part 10.1 - Known open port



```
# Part 10 — Robustness Tests

Demonstrate at least **four** of the following:

1. A known open port
2. A known closed/not-open port
3. Invalid hostname
4. Non-numeric port input
5. Port below 1
6. Port above 65535
7. Start port greater than end port
8. No open ports in the selected range
9. A connection attempt that times out

Your program should handle expected errors gracefully rather than terminating with an unhandled traceback.

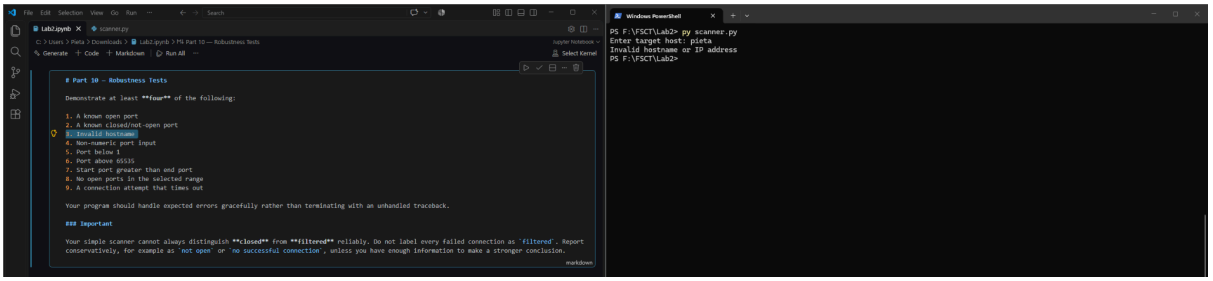
### Important

Your simple scanner cannot always distinguish **closed** from **filtered** reliably. Do not label every failed connection as `filtered`. Report conservatively, for example as `not open` or `no successful connection`, unless you have enough information to make a stronger conclusion.
```

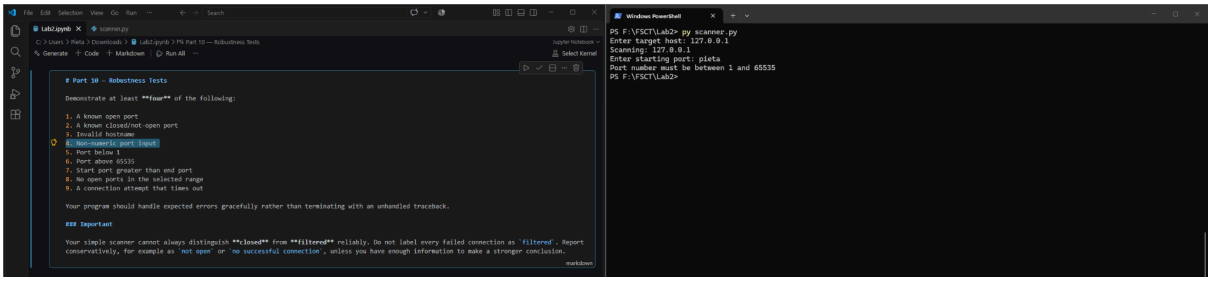
```
PS F:\FSCT\Lab2> py scanner.py
Enter target host: 127.0.0.1
Scanning: 127.0.0.1
Enter starting port: 20
Enter ending port: 100
Port 80 Service: http
Open ports: [80]
PS F:\FSCT\Lab2>
```

```
C:\Users\Pieta>
C:\Users\Pieta>py -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
```

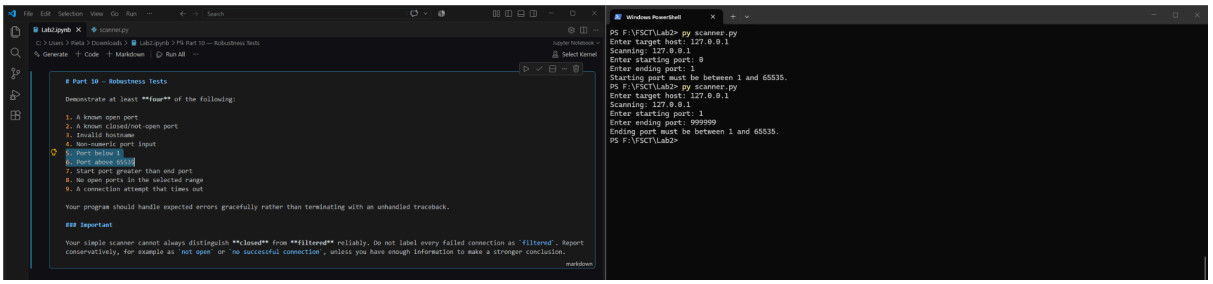
Part 10.2 - Invalid hostname



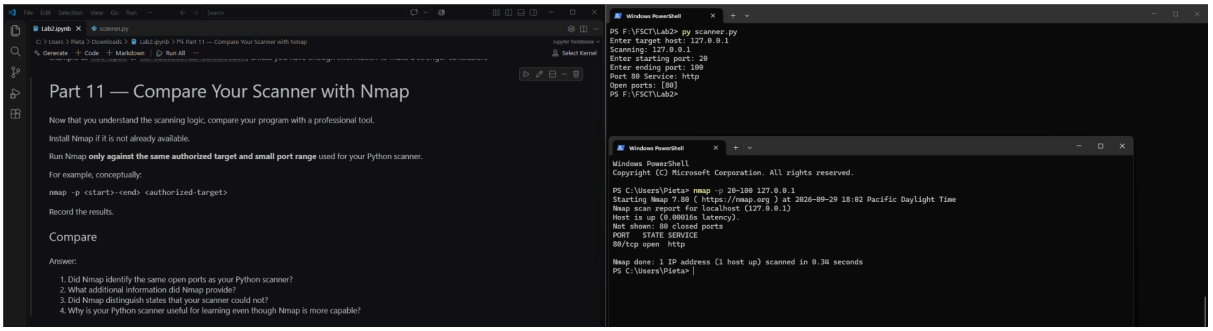
Part 10.3 - Non-numeric port input



Part 10.4 - Port below 1 and Port above 65535



Part 11



1. Did Nmap identify the same open ports as your Python scanner?

Yes it did

2. What additional information did Nmap provide?

The host status, latency, number of closed ports, and scan completion time.

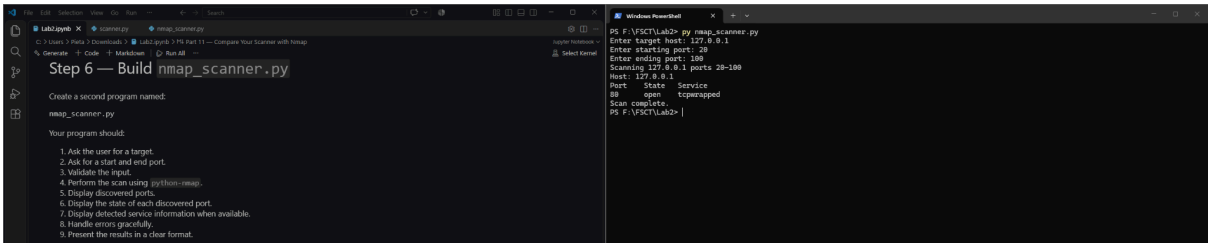
3. Did Nmap distinguish states that your scanner could not?

Yes. Nmap can distinguish between states such as open, closed, and filtered

4. Why is your Python scanner useful for learning even though Nmap is more capable?

My scanner is useful because it shows how port scanning works using sockets and TCP connection attempts.

Part 12



Part 12 Questions:

1. Which version required more code?

The raw socket scanner required more code.

2. Which version provided more information?

The python-nmap scanner provided more information.

3. What information did Nmap provide that your raw socket scanner did not?

Nmap provided more detailed port states and host information.

4. What work is `python-nmap` hiding from the programmer?

It hides the low level scanning and connection process.

5. Why was it useful to build the raw socket version **before using `python-nmap`?**

To understand how port scanning works.

6. If you were developing a real security application, which approach would you choose and why?

Nmap, because it provides more detailed information and features.

Part 13 Questions:

1. What information can an attacker learn from port scanning?

To find open ports and running services.

2. Why can an open port increase a system's attack surface?

It provides another possible entry point into a system.

3. Does an open port automatically mean that a vulnerability exists? Explain.

No. It only means a service is accepting connections.

4. Why should unnecessary network services be disabled or restricted?

To reduce possible entry points for attackers.

5. What is the difference between an open port and a closed port?

An open port accepts connections; a closed port does not.

6. Why might a firewall make scan results difficult to interpret?

It can block or filter connections to ports.

7. Why can our simple TCP Connect scanner not reliably identify every filtered port?

It mainly detects whether a connection succeeds or fails.

8. How could defenders detect or limit scanning activity?

Firewalls, monitoring, and disabling unnecessary services.

9. Why should defenders scan their own systems?

To find exposed ports and unnecessary services.

10. Why is authorization important before performing network scanning?

Scanning without permission can violate rules or laws.

Part 14 Analysis

In this lab, I created a basic TCP port scanner in python and compared it to nmap. I scanned my local host address (127.0.0.1) using ports 20–100. I hosted a simple HTTP server on port 80 to test with. My python scanner found that port 80 was open and identified with the service as HTTP. I then scanned the same target and port range using nmap, which also found port 80 open with HTTP.

Both scanners found the same open port, however, nmap gave me more information about the scan. It showed that the host was up, the latency, the number of closed ports, and how long the scan took. My python scanner was much simpler and only displayed the ports that were open. It did not show any additional information.

Having an open port does not mean that there is a vulnerability. It just means that a service is running and accepting connections on that port. However, having more open ports can increase the attack surface because there are more services that someone could potentially target. If a service is outdated, misconfigured, or not needed, it could create a security risk.

Port scanning can also be useful for defenders. An administrator can scan their own systems to see what ports are exposed and check if any unnecessary services are running. Firewalls can also be used to block or filter unwanted connections, while services that are not needed should be disabled.

Overall, this lab helped me understand how port scanning actually works instead of only using a tool like nmap. Building the python scanner showed me how sockets can be used to test whether ports are open. nmap showed how a more advanced tool can do the same thing while also providing more detailed information about the scan.